MIDDLESEX UNIVERSITY

THE BURROUGHS, HENDON LONDON NW4 4BT

TITLE: CST2550 PROJECT MANAGEMENT REPORT

KAY BUGINGO: M00971312

NIGEL GAMBIZA: M00951628

CHOTA FIKANSA CHANDA: M01034342

ROHAN HAULKHORY: M01004090

KOSHAL MOHESS: M00943654

Table of Contents

1. Project Description 3
2. Team Roles and Responsibilities 3
3. Tools and Collaboration Platforms 4
   - Version Control: GitHub 4
   - Project Management: GitHub Projects 4
   - Communication: WhatsApp 4
   - Development Environment: Visual Studio 4
   - Database: SQLite via Microsoft.Data.Sqlite 5
   - Testing: MSTest 5
   - Diagrams: draw.io, canva, lucid 5
4. Meeting Notes 5
   - Ongoing Communication: WhatsApp 6
5. Workload Division 6
   - Work Breakdown Structure 7
6. Testing Management 7
   - What Was Tested 7
   - How Bugs Were Reported and Fixed 7
   - How Testing Aligned with Development Sprints 7
7. Risk Management 8
8. Project Design and Implementation Management 8
   - Use Case Diagram 9
   - System Flowchart 10
   - Admin Flowchart 10
   - Customer / User Flowchart 11
   - AI Agent Flowchart 12
   - Login Flowchart 12
9. Gantt Chart 13

1. Project Description

Group 5 was tasked with designing, developing, and testing a software system for a takeaway restaurant. The overall aim was to build a functional management system that allows restaurant staff to efficiently handle menu operations and customer orders in a fast-paced environment.

The system was required to store and retrieve menu data using a custom-built data structure, persist data to a database, and provide a user interface through which staff could interact with the system. An additional AI agent component was introduced for assistance, by providing a chat interface for the user's menu navigation.

The key objectives the team needed to achieve were:

- Design and implement a custom Binary Search Tree to store and manage menu items.
- Integrate a SQLite database for persistent data storage.
- Build a web-based interface supporting two user roles: Admin and Customer.
- Implement an AI agent accessible to customers through a chat interface.
- Write unit tests to verify the correctness of the core data structure.
- Manage the project using Agile/Scrum methodology with defined team roles.

The project was developed over 8 sprints between 4th March and 10th April 2025, version control managed through Github.

2. Team Roles and Responsibilities

Roles were assigned at the start of the project based on each member's strengths and the requirements set out in the coursework brief. The five roles assigned were Scrum Master, Secretary, two Developers and a Tester.

Scrum Master – Kay

Responsible for project coordination, sprint planning, and ensuring the team stayed aligned throughout the development process. Key responsibilities included setting up the GitHub repository, maintaining the GitHub Projects board, tracking task progress across sprints, facilitating team meetings, coordinating design decisions between team members, and leading the report writing. The Scrum Master also contributed directly to the technical work by co-designing the SQL schema and C# classes alongside the lead developer – Nigel.

Secretary – Koshal

Responsible for documenting sprint meetings and maintaining formal records of team decisions. The Secretary recorded meeting minutes for the sprint meetings and ensured that key decisions discussed over WhatsApp were documented.

Developer 1 – Nigel

Responsible for the core backend implementation. Nigel implemented the AI agent independently, co-designed the SQL database schema, and co-defined the C# data classes. Nigel also developed the *TakeawayWeb ASP.NET* Core MVC, including the web controllers, views, and the overall web application structure.

Developer 2 – Chota

Responsible for algorithm design and class structure. Chota wrote the pseudocode for the insert, remove, and search algorithms, and co-designed the class diagrams. Chota also contributed to the implementation of the core console application logic, before it was then re-evaluated to a web application using ASP.NET Core MVC.

Tester – Rohan

The Tester set up the *MSTest* unit test project, wrote and executed the test cases covering the *BinarySearchTree* operations, and reported any issues found back to the developers for resolution.

Accountability

Progress was tracked through the Github Projects board (issues section), which was maintained by the Scrum Master throughout the project. Each task was assigned to a specific team member as a GitHub issue, allowing individual contributions to be clearly visible in the project history. Code contributions were tracked through GitHub commit history, with each member committing their own work directly to the repository.

3. Tools and Collaboration Platforms

Version Control: GitHub

GitHub was used as the primary version control system for the project. All source code, diagrams, and documentation were committed to a single public repository, ensuring all team members always had access to the latest version of the project.

GitHub was selected over Azure DevOps Repos because it offered free repository hosting without requiring a paid subscription, which was a practical constraint for the team. Commits were made directly to the main branch, with each member responsible for committing their own work.

Project Management: GitHub Projects

GitHub Projects was used to manage the team backlog and track task progress across sprints. Tasks were created as GitHub Issues and assigned to individual team members as work progressed. GitHub Projects was chosen over Azure DevOps Boards because it integrates directly with the GitHub repository, keeping code and task management in one place.

Communication: WhatsApp

WhatsApp was used for regular team communication. This was used for sharing progress, coordination decisions between sprints, distributing work between team members. WhatsApp was chosen because all team members were already active on the platform.

Development Environment: Visual Studio

Microsoft Visual Studio was used as the IDE for C# coding. Visual Studio was selected because it provides native support for ASP.NET Core, MSTest, and SQLite.

Database: SQLite via Microsoft.Data.Sqlite

SQLite was used as the database engine, accessed through the Microsoft.Data.Sqlite NuGet package. It was chosen for its lightweight, file-based nature, requiring no server installation or configuration.

Testing: MSTest

The MSTest framework was used for unit testing, integrated directly into Visual Studio's Test Explorer. MSTest was selected because it is the default testing framework supported by Visual Studio.

Diagrams: draw.io, canva, lucid

These platforms were used to create the use case diagram, flowcharts, and class diagrams. They were chosen because they are free, browser-based, and support *export to PNG* for easy inclusion in the repository.

4. Meeting Notes

The team held two formal sprint meetings over the course of the project. Because of team members' availability issues, communication and decision-making took place continuously via a WhatsApp group.

Meeting 1: Sprint Planning – **Date**: 31st January 2026 – **Attendees**: Chota, Kay, Nigel, Rohan – **Format**: Whatsapp voice call.

Agenda and Decisions:

- Roles were formally assigned: Scrum Master (Kay), Secretary (Koshal), Developer 1 (Nigel), Developer 2 (Chota), Tester (Rohan)
- The project theme was confirmed as a Takeaway Restaurant System
- The team agreed to use GitHub for version control and WhatsApp for communication
- Initial backlog items were created on GitHub Projects
- The application type was discussed — the team agreed to begin with a console application and evaluate further as development progressed
- Tasks for Sprint 1 were assigned: class design, data structure research, and SQL schema planning

Meeting 2 : Sprint Review and Progress - Date: 6th March 2026 **Attendees:** All five team members **Format:** Whatsapp voice call.

Agenda and Decisions:

- Progress was reviewed against the Sprint 1 backlog
- The console application was demonstrated — core BST operations (insert, delete, search) and database integration were confirmed working
- The team agreed to have the application run on WinForms instead of console.
- Unit test project setup was assigned to the Tester

- Report writing responsibilities were discussed and assigned to the Scrum Master with contributions from all members

Ongoing Communication: WhatsApp

Between formal meetings, the team communicated daily through WhatsApp. Key decisions made via WhatsApp during the project included:

- Agreement on the Binary Search Tree as the primary data structure following a group discussion initiated by Developer 1 (Nigel)
- Decision to adopt ASP.NET Core MVC over WinForms, driven by the requirements of the AI agent feature
- Coordination of code integration between the console app and web app components
- Review and feedback on report sections as they were drafted

5. Workload Division

Tasks were allocated based on each team member's role, strengths, and the technical requirements of the project. The Scrum Master coordinated the overall distribution, ensuring no single member was overloaded and that dependencies between tasks were managed - for example, the database schema needed to be agreed before the *DatabaseHelper* class could be implemented, and the BST needed to be complete before the Tester could write meaningful unit tests.

Team Member	Role	Key Contributions
Kay	Scrum Master	Project coordination, GitHub Projects board maintenance, SQL schema co-design, C# class co-design, class diagram co-design, report writing for both reports
Nigel	Developer	AI agent implementation, *TakeawayWeb* MVC application, web controllers and views, SQL schema co-design, C# class co-design
Chota	Developer	Pseudocode for insert/remove/search algorithms, class diagram co-design, console application logic, testing alongside the tester
Rohan	Tester	MSTest unit test project setup, unit test writing and execution, bug reporting
Koshal	Secretary	Sprint meeting documentation

Progress was tracked through the GitHub Projects board, updated regularly by the Scrum Master. Tasks moved through five stages: *Backlog, Sprint Backlog, In Progress, In Review*, and *Done*. progress was communicated through WhatsApp, where members would flag when tasks were completed or when they were blocked.

Work Breakdown Structure

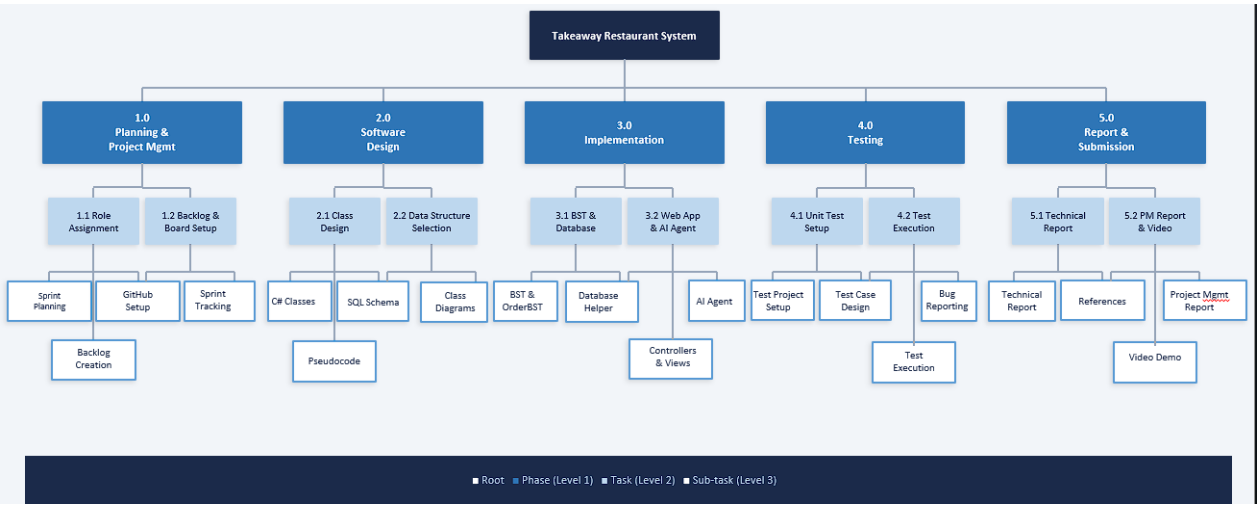

Figure 1: Work Breakdown Structure

6. Testing Management

The Tester was responsible for setting up the MSTest unit test project, designing and writing all test cases, executing the tests, and reporting any failures back to the developers.

What Was Tested

Unit tests focused on the core custom data structure — the *BinarySearchTree* — covering insert, delete, search by ID, search by name, in-order traversal, and count operations. Tests were designed to cover valid inputs, invalid inputs, edge cases, duplicate entries, and non-existent search targets.

How Bugs Were Reported and Fixed

When a test failed, the Tester reported the issue directly to the relevant developer via WhatsApp, describing the failing test case and the observed output. The developer investigated and applied a fix, after which the Tester re-ran the affected tests to confirm the issue was resolved. All 12 unit tests passed on final execution.

How Testing Aligned with Development Sprints

Unit test development began once the BinarySearchTree class reached a stable state, approximately midway through the project. This allowed the Tester to write meaningful tests against real implementations, and the test project was maintained in the same GitHub repository as the source code.

7. Risk Management

Risk	Type	Likelihood	Impact	Mitigation
BST degrades to O(n) if items inserted in sorted order	Technical	Low	Medium	Data inserted in non-sequential order in practice; acknowledged as known limitation
Application type changed late in the project	Technical / Time	Occurred	High	Team realigned quickly via WhatsApp; existing BST logic was reused in web app
Fixed-size order array limits scalability	Technical	Low	Low	Accepted as a simplification for academic scope; noted as limitation in report
Hardcoded database path in web controllers	Technical	Occurred	Medium	Identified by Scrum master and quickly flagged for correction; eventually dealt with by Developer (Nigel)
Insufficient meeting documentation	Process	Occurred	Medium	WhatsApp conversation logs retained; two formal meetings documented
Build artefacts committed to repo	Process	Occurred	Low	Identified late; .gitignore to be configured before final submission
Team members unfamiliar with ASP.NET Core MVC	Resource	Medium	Medium	Nigel led web app development; other members contributed to areas they were confident in

8. Project Design and Implementation Management

How Design Decisions Were Coordinated

Design decisions were made across the team, with discussions initiated via WhatsApp and formalised during sprint meetings. The Scrum Master coordinated the design direction, ensuring that decisions made by members were communicated to the rest of the team before implementation began.

How Progress Was Tracked

Progress was monitored through the GitHub Projects board, with tasks moving through Backlog → Sprint Backlog → In Progress → In Review → Done as work progressed. The Scrum Master updated the board regularly and used it as the basis for progress reviews during sprint meetings and WhatsApp check-ins.

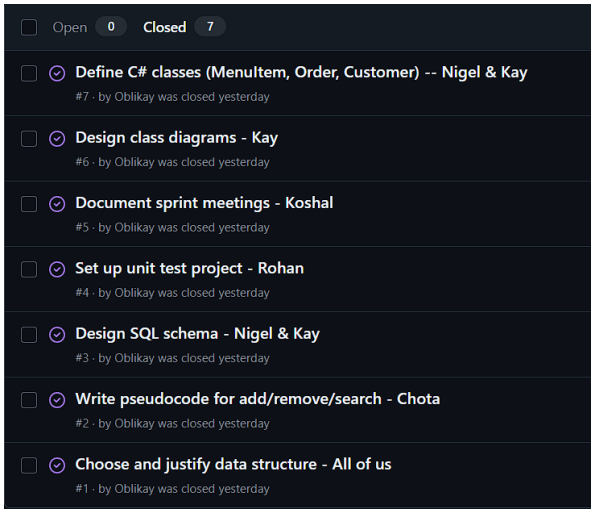

Figure 2: Progress Tracker on GitHub Projects

Use Case Diagram

The use case diagram below defines the two primary actors in the system — Admin and Customer — and their respective interactions. Admin users can manage the menu and orders, while Customer users can browse the menu and place orders.

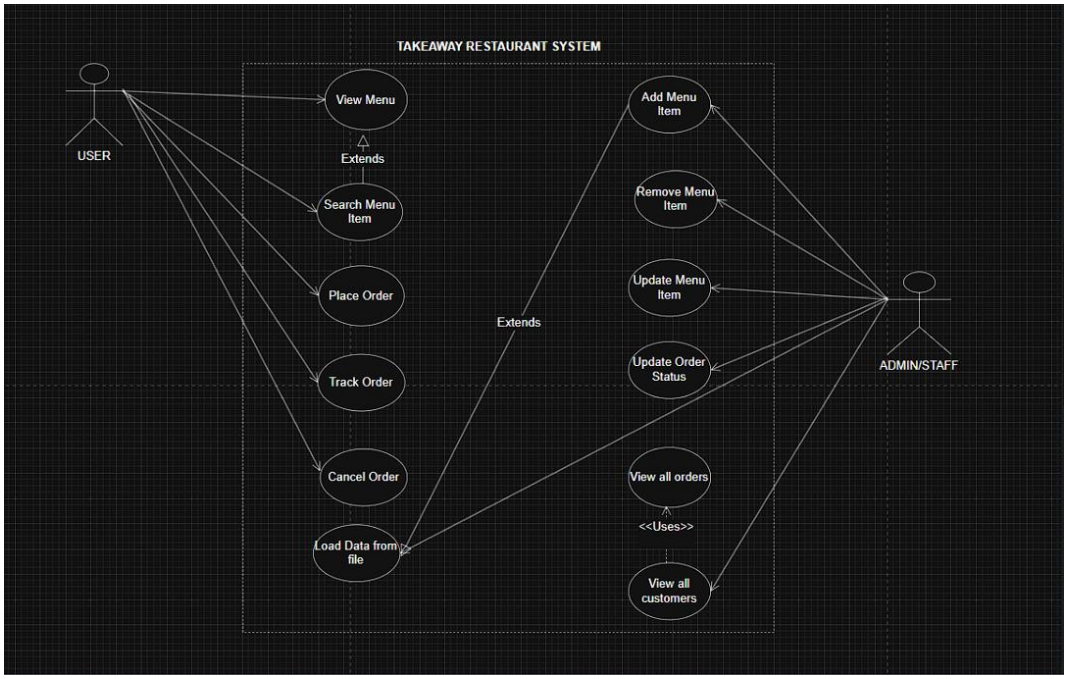

Figure 3: Use Case Diagram

System Flowchart

The flowchart below shows the system flow, from user loging through to the key actions available to each role.

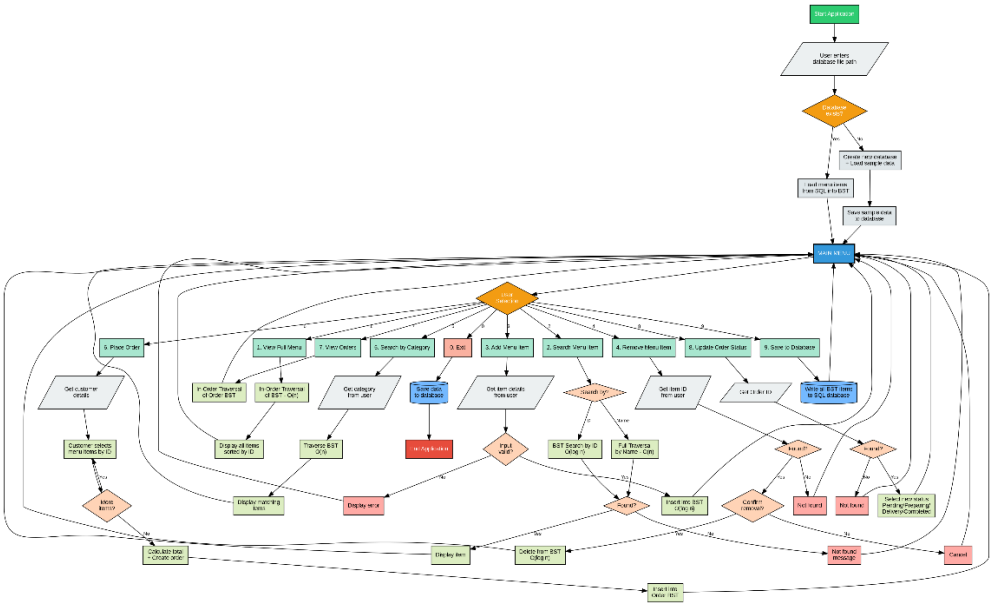

Figure 4: System Flowchart

Admin Flowchart

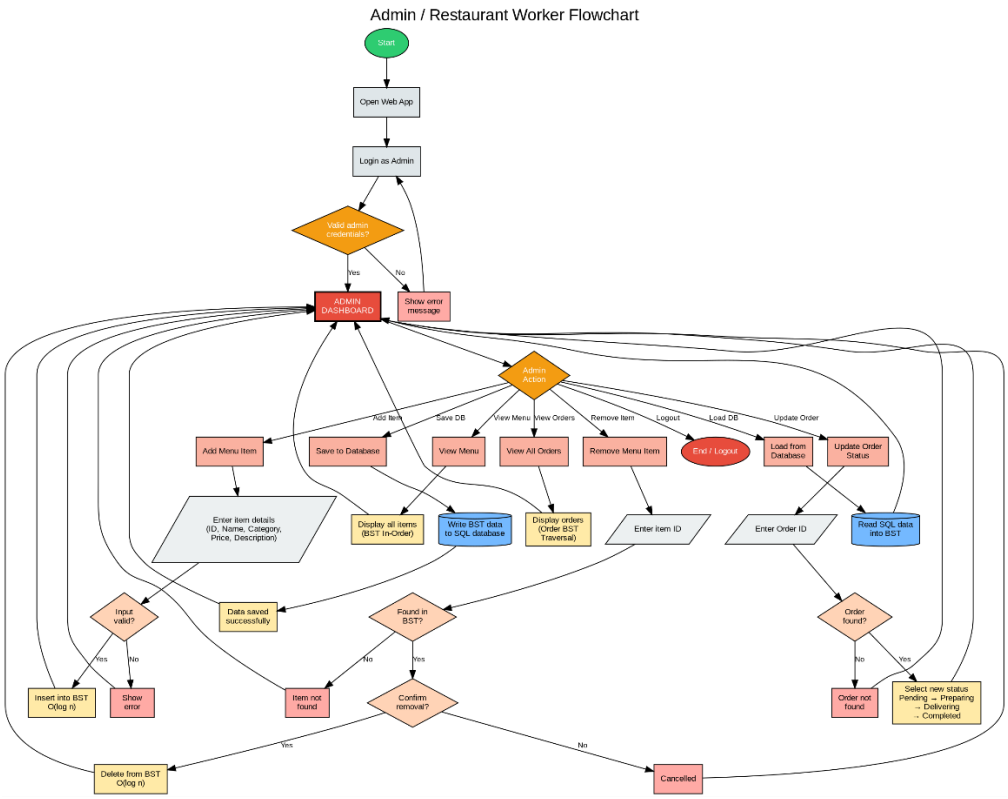

Figure 5: Admin Flowchart

Customer / User Flowchart

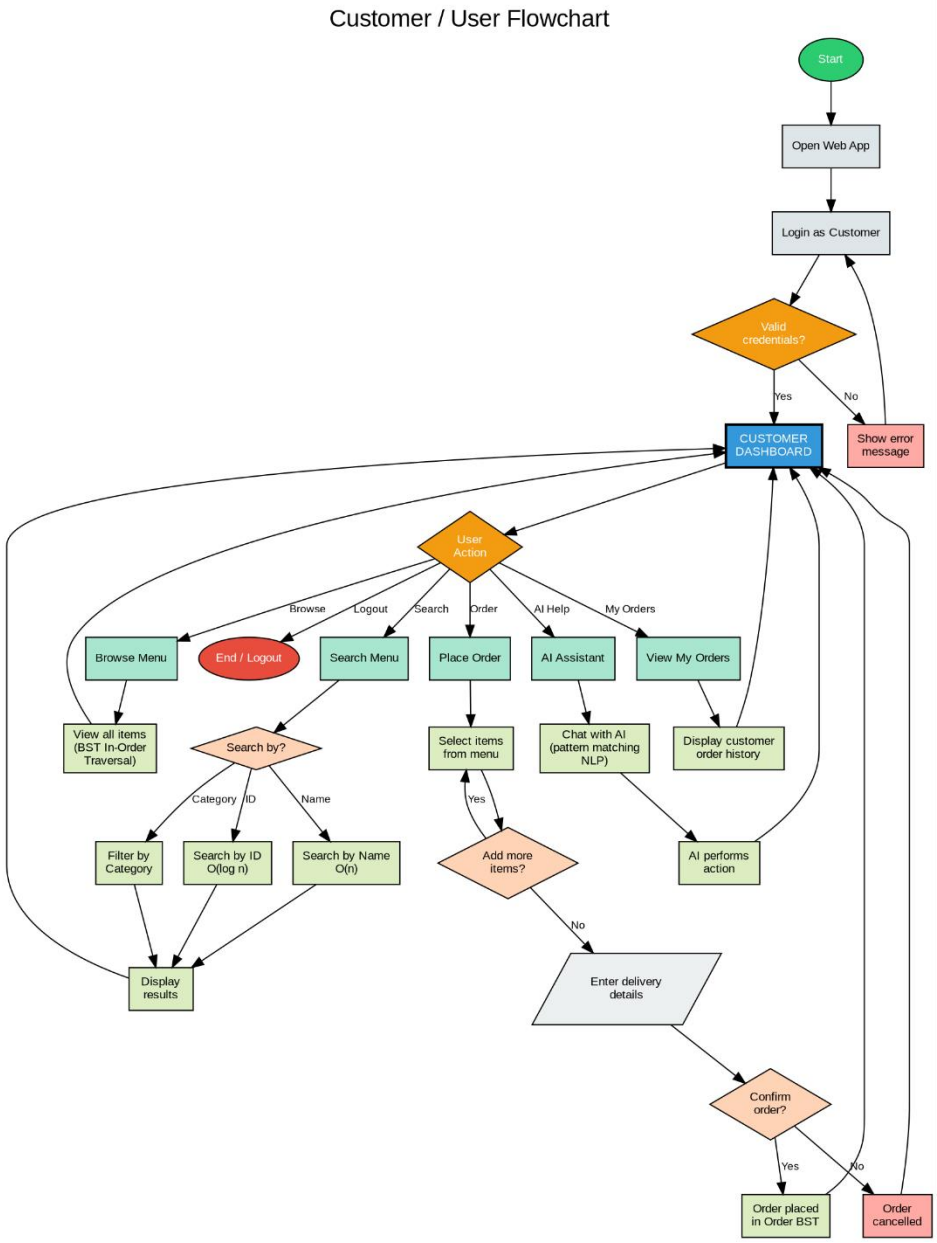

Figure 6: Customer/User Flowchart

AI Agent Flowchart

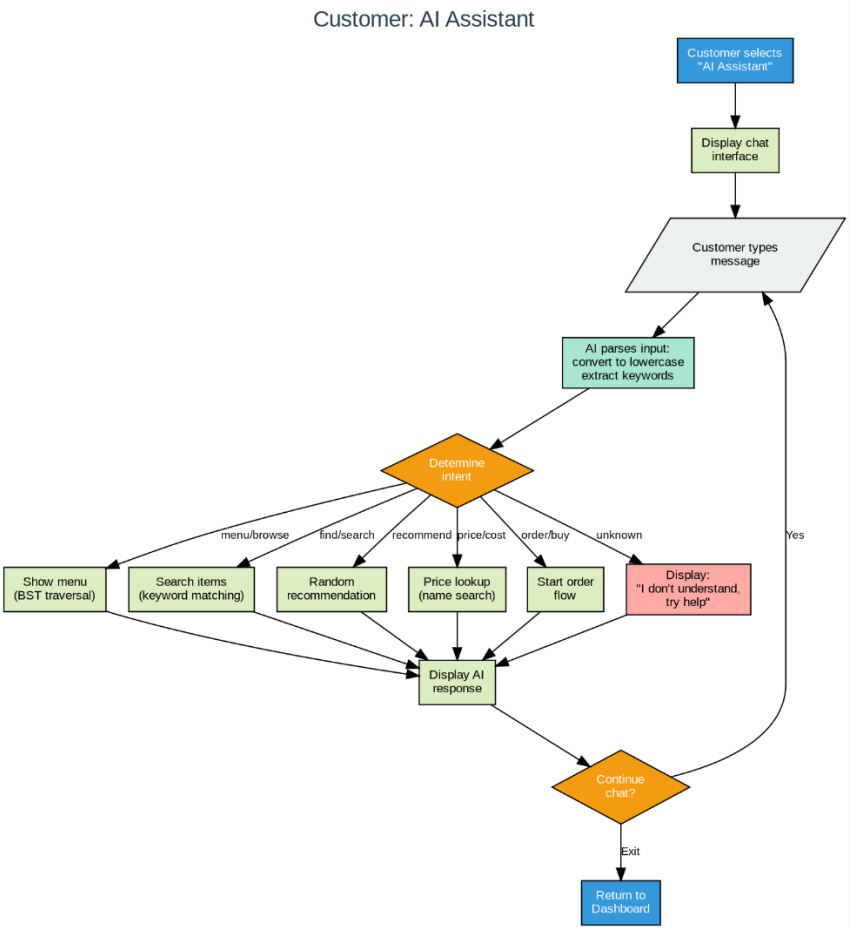

Figure 7: AI Agent Flowchart

Login Flowchart

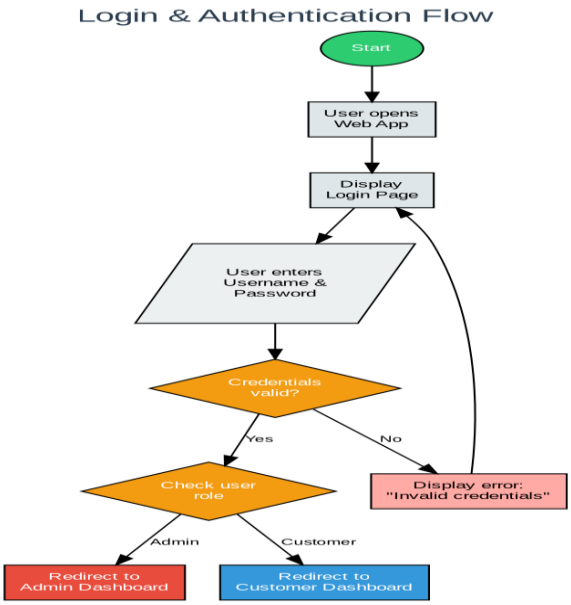

Figure 8: Login Flowchart

9. Gantt Chart

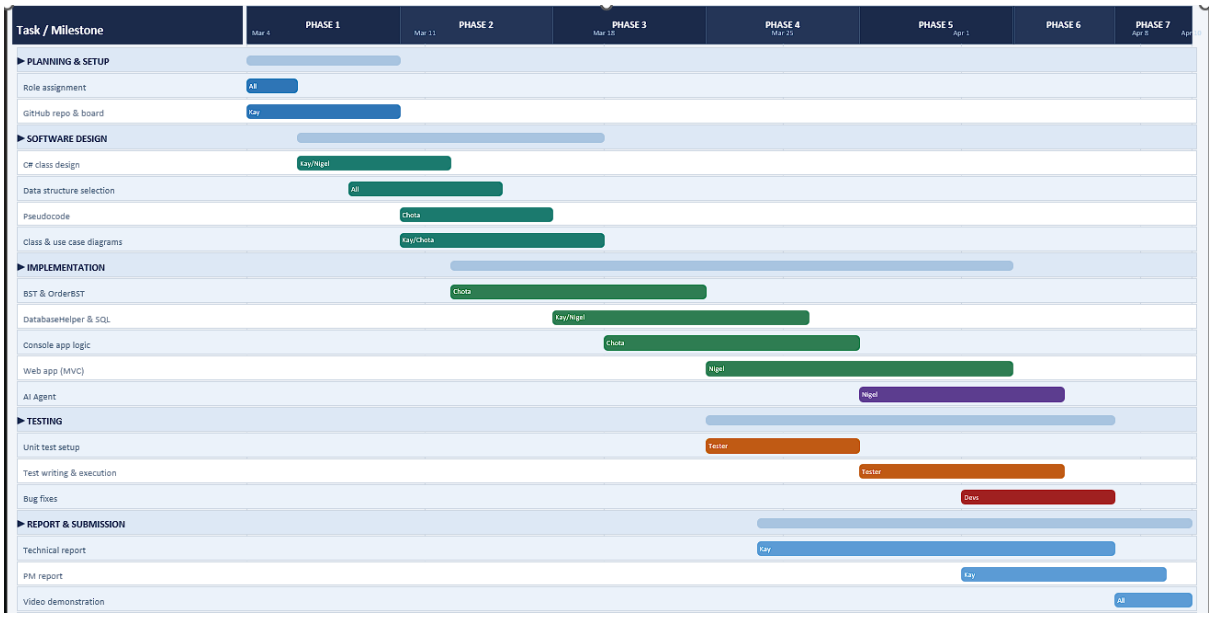

Figure 9: Gantt Chart